

Workshop for Economic Modeling in Julia

Y.-H. Henry Chen

Research Scientist, MIT CS3

March 31, 2026

Any name coincidence is *purely accidental*...

John created a CGE model “M21i.jl” using MPSGE.jl for policy simulations. He would like to make sure other modelers can replicate his results, and his team members can continue to add new components into the model without potentially introducing unexpected code conflicts due to package version issues. What can he do?

Create a project environment

- Create a project environment for M21i.jl

- In this example, M21i.jl is in D:\work\temp\workshop\
 - **Goal:** turn the folder “workshop” into a project environment
 - In VS code => **File** => **Open Folder** => select folder “workshop”
 - **Terminal** => **New Terminal** => Ctrl + shift + p => **Julia: Start REPL**
 - Under “Julia >” => **]** => you will see something like “(@v1.12) pkg>”

Julia mode

Package manager mode

Current active environment: default global environment for Julia 1.12

- Under “pkg >” => **activate .** => you will see “(workshop) pkg>”

Current active environment: workshop

- Back to Julia mode “Julia >” => **using Pkg** => **Pkg.status()** => No dependencies for now

Create a project environment

- M21i.jl in the project environment “workshop”
 - Open M21i.jl => run the three lines below:

```
using Pkg  
Pkg.add("JuMP")  
Pkg.add("MPSGE")
```

=> Project.toml and Manifest.toml are created automatically

Create a project environment

- M21i.jl in the project environment “workshop”

- Project.toml — names & identities of the direct dependencies of M21i.jl

```
1  [deps]
2  JuMP = "4076af6c-e467-56ae-b986-b466b2749572"
3  MPSGE = "d5dc2f44-7ae2-49e9-bc77-b47b6bca565d"
```

- Manifest.toml — the dependency graph, dependency version, where to load

```
Entry for JuMP ← 190  [[deps.JuMP]]
Dependencies of JuMP ← 191  deps = ["LinearAlgebra", "MacroTools", "MathOptInterface", "MutableArithmetics"]
Pin JuMP to a specific commit ← 192  git-tree-sha1 = "4091a1338a0e32766b11b9bd3fac247d34200c77"
Unique identifier of JuMP ← 193  uuid = "4076af6c-e467-56ae-b986-b466b2749572"
Version number ← 194  version = "1.30.0"
271  [[deps.MPSGE]]
272  deps = ["DataFrames", "JuMP", "OrderedCollections", "PATHSolver"]
273  git-tree-sha1 = "c41433d82612252b090300f38af9259436394ae2"
274  uuid = "d5dc2f44-7ae2-49e9-bc77-b47b6bca565d"
275  version = "0.6.4"
```

Different packages may have the same name and it's fine as long as they reside in different environments.

Create a project environment

- M21i.jl in the project environment “workshop”

- Add the following two lines (lines 6–7)

```
3 using Pkg
4 Pkg.add("JuMP")
5 Pkg.add("MPSGE")
6 Pkg.activate(@__DIR__)
7 Pkg.instantiate()
8 using JuMP, MPSGE
```

→ Activate the project environment at where M21i.jl locates

→ Install and set up all packages required by the current project environment

- Let's comment out lines 4, 5, & 7 and see what happens next time when M21i.jl is reopened

- After saving M21i.jl and leaving VS code, let's turn on VS code again, and open M21i.jl
- Running M21i.jl, line 6 will activate the project environment
- Pkg.instantiate() is not required when packages have been installed locally
- Pkg.instantiate() is needed when you clone or download someone's project

Create a project environment

- Advantages of working in a project environment
 - Reproducibility
 - ensures exact versions of packages used
 - `Pkg.add()` will not reinstall packages exist locally
 - Relying only on `Pkg.add()` without having a project environment can be problematic. Why?
 - When code is run elsewhere w/o required packages, newer version packages may be installed
 - So, reproducibility may not be achievable
 - Without having a project environment, package version conflicts may happen

Create a project environment

- Advantages of working in a project environment
 - Isolation
 - When trying another package versions for your project, simply create a new environment for that
 - Each project only adds packages it needs
 - Developments by collaborators can use the same project environment—the same dependency versions
 - Portability
 - Script can be run on various machines under the same project environment by having:
 - `Pkg.activate(@__DIR__)` —> Turn the folder where the script file is into a project environment
 - `Pkg.instantiate()` —> Project.toml & Manifest.toml are read to download and install any missing packages

Plan ahead...

More things to be covered. Let me know if you are interested in, and I can plan to introduce more advanced materials later!

Bibliography

- Bezanson, J., Edelman, A., Karpinski, S., & Shah, V. B. (2017). Julia: A fresh approach to numerical computing. SIAM Review, 59(1), 65-98. <https://doi.org/10.1137/141000671>